

EMPLOYEE PRODUCTIVITY ANALYSIS

A PROJECT REPORT

Submitted by

RAVI TEJA E	110521104308
BHAKKIYA LAKSHMI P	110521104007
ASMITHA E	110521104006

*In partial fulfillment of the requirements for the degree
of*

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

**GOJAN SCHOOL OF BUSINESS AND TECHNOLOGY
EDAPALAYAM, REDHILLS, CH-52**

ANNA UNIVERSITY: CHENNAI 600 025

APRIL/MAY 2025

BONAFIDE CERTIFICATE

Certified that this project report “**EMPLOYEE PRODUCTIVITY ANALYSIS**” is the bona fide work of **RAVI TEJA E (110521104308)**, **BHAKKIYA LAKSHMI P (110521104007)**, **ASMITHA E (110521104006)**, who carried out the project work under my supervision. I further believe that, to the best of my knowledge, the work reported here is not a part of any other project work.

SIGNATURE

HEAD OF DEPARTMENT

Mrs. REVATHI. L, M.E.,
Department of Computer Science and
Engineering,
Gojan School of Business and
Technology,
80 Ft road, Edapalayam,
Redhills, Chennai-52

SIGNATURE

SUPERVISOR

Mrs. REVATHI. L, M.E.,
Department of Computer Science
and Engineering,
Gojan School of Business and
Technology,
80 Ft road, Edapalayam,
Redhills, Chennai-52

Submitted for the Project Viva Voice Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

COLLEGE NAME Gojan School of Business & Technology

DEGREE/BRANCH B.E. / Computer Science and Engineering

SEMESTER : VIII

NAME OF THE STUDENTS : RAVI TEJA E (110521104308)
BHAKKIYA LAKSHMI P
(110521104007)
ASMITHA E (110521104006)

TITLE OF THE PROJECT EMPLOYEE PRODUCTIVITY
ANALYSIS
.

SUPERVISOR Mrs. REVATHI. L, M.E.,

The report of this project has been submitted by the above student in partial fulfillment for the award of the degree **Bachelor of Engineering in Computer Science and Engineering** from **Anna University**, Chennai. The report has been evaluated, and the authenticity of the work has been confirmed.

Submitted for the University Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

RAVI TEJA E

Department of Computer Science and Engineering
Gojan School of Business and Technology

DECLARATION

I hereby declare that the project entitled “Employee Productivity Analysis” submitted by us to Anna University in May/June 2025 for the award of the degree of Bachelor of Engineering in Computer Science and Engineering is the original and independent teamwork carried out by me under the guidance of **Mrs. REVATHI. L, M.E.**, Assistant professor, and that it has not been formed the basis fully or partially for the award of any degree, diploma, or other similar titles earlier, and no part of the above project work has been published or sent for publication at the time of submission.

Place: Chennai

Signature

Date: -

BHAKKIYA LAKSHMI P

Department of Computer Science and Engineering
Gojan School of Business and Technology

DECLARATION

I hereby declare that the project entitled “Employee Productivity Analysis” submitted by us to Anna University in May/June 2025 for the award of the degree of Bachelor of Engineering in Computer Science and Engineering is the original and independent teamwork carried out by me under the guidance of **Mrs. REVATHI. L, M.E.**, Assistant professor, and that it has not been formed the basis fully or partially for the award of any degree, diploma, or other similar titles earlier, and no part of the above project work has been published or sent for publication at the time of submission.

Place: Chennai

Signature

Date: -

ASMITHA E

Department of Computer Science and Engineering
Gojan School of Business and Technology

DECLARATION

I hereby declare that the project entitled “Employee Productivity Analysis” submitted by us to Anna University in May/June 2025 for the award of the degree of Bachelor of Engineering in Computer Science and Engineering is the original and independent teamwork carried out by me under the guidance of **Mrs. REVATHI. L, M.E.**, Assistant professor, and that it has not been formed the basis fully or partially for the award of any degree, diploma, or other similar titles earlier, and no part of the above project work has been published or sent for publication at the time of submission.

Place: Chennai

Signature

Date: -

ACKNOWLEDGEMENT

We express our deepest gratitude to our **Chairman, Dr. G. Natarajan, Ph.D.**, and **Chairperson Mrs. Brindha Natarajan, B.Com**, for their valuable guidance and blessings.

We are deeply indebted to our beloved **Principal, Dr. C. Selvakumar, Ph.D.**, Gojan School of Business and Technology, for providing us with an excellent environment in which to complete our course.

We express our sincere thanks to our **Head of the Department, Mrs REVATHI L, M.E.**, Assistant Professor, who has been a constant source of inspiration and guidance throughout the project.

We thank our Supervisor, Mr PRABAKARAN, I, M.E., Assistant Professor, for his exemplary guidance in completing our project.

We thank all the **Staff Members** of our department for their valuable support and assistance at various stages of our project development.

Finally, we take this opportunity to extend our deep sense of gratitude and appreciation to our family and friends for all that they meant to us during the crucial times of the completion of our project.

Employee Productivity Analysis

ABSTRACT

In the increasingly competitive global business landscape, maximizing employee productivity is not only essential for the sustained success of organizations but also critical for their ability to adapt to rapidly changing market conditions. However, traditional methods of tracking and evaluating employee performance, such as manual reports, subjective reviews, and performance appraisals, often fail to provide accurate, objective, and real-time feedback. These conventional methods also tend to overlook significant factors influencing productivity, leading to inefficiencies in workforce management and resource allocation. This project, titled *Employee Productivity Analysis*, aims to address these challenges by offering a comprehensive, data-driven solution that leverages advanced **machine learning** techniques to monitor, analyze, and predict employee productivity.

The core objective of the system is to enable organizations to gain a deeper understanding of employee performance by integrating **task-based monitoring**, **automated attendance tracking**, and **quantitative performance metrics** into a unified framework. By continuously tracking and analyzing key data points—such as task completion rates, working hours, deviations from estimated deadlines, and the efficiency of time spent on different tasks—the system generates accurate, real-time productivity scores for each employee.

At the heart of the system is a **Gradient Boosting machine learning model**, which is trained on historical and real-time employee data to predict productivity levels. This model applies sophisticated algorithms that can identify patterns in employee behavior, task performance, and time management, ensuring that predictions are robust and accurate..

TABLE OF CONTENTS

1. Abstract	1
2. List of Abbreviations	2
3. Chapter 1 – Overview	3
1.1 System Overview	3
1.2 Project Overview	5
4. Chapter 2 – Algorithms	7
2.1 Long Short-Term Memory (LSTM)	7
2.2 Gradient Boosting Algorithm	8
2.3 Support Vector Machine (SVM)	9
2.4 Logistic Regression	10
5. Chapter 3 – Existing System	11
3.1 Gaps in Current Solutions	11
6. Chapter 4 – Proposed System	13
4.1 Need for a New System	13
4.2 Implementation Workflow	14
4.3 Key Features and Innovations	17
7. Chapter 5 – Literature Survey	19
5.1 Data Collection	19
5.2 Machine Learning Approach	21
8. Chapter 6 – Project Modules	23
6.1 System Components	23
6.2 Predictive and Prescriptive Analytics	26
9. Chapter 7 – System Design & Architecture	28
7.1 Architecture Diagram	28
7.2 Use Case Diagram	29
7.3 Class Diagram	30
7.4 Activity Diagram	31
7.5 Sequence Diagram	32

7.6 Entity Relationship Diagram	33
7.7 Collaboration Diagram	34
10. Chapter 8 – System Requirements	35
8.1 Technologies Used	35
8.2 Core Backend Functionalities	36
8.3 General Requirements	37
8.4 Environment, Hardware, Software	38
11. Chapter 9 – Source Code & Program Structure	43
10.1 Employee Dashboard	43
10.2 Main File	44
10.3 Login Page	44
10.4 Predict Employee Performance	45
10.5 Register Page	46
10.6 Result Page	46
10.7 Backend (app.py)	47
10.8 Model Training (training_model.py)	48
10.9 Program Structure	48
12. Chapter 10 – Output Screenshots	49
11.1 Welcome Page	49
11.2 Login Form	49
11.3 Register Page	50
11.4 User Interface	50
11.5 Punch-in & Punch-out Page	51
11.6 Performance Analytics	51
11.7 Output Sample	52
13. Chapter 11 – Conclusion & Future Enhancements	40
9.1 Summary of Key Findings	40
9.2 Impact on Workforce Management	41
9.3 Future Enhancements	42
14. References	53

LIST OF ABBREVIATIONS

- **ML** – Machine Learning
- **LSTM** – Long Short-Term Memory
- **API** – Application Programming Interface
- **UI** – User Interface
- **HR** – Human Resources
- **KPI** – Key Performance Indicator
- **PS** – Productivity Score
- **Et** – Task Efficiency Score
- **Ac** – Attendance Consistency
- **E.g.**, Engagement Score
- **Z** – Z-Score
- **MAC** – Machine Access Control (*assumed*)
- **SFOF** – System for Operational Functions (*assumed*)
- **GMS** – Global Management System (*assumed*)
- **CSV** – Comma-Separated Values
- **ROC-AUC** – Receiver Operating Characteristic – Area Under Curve
- **F1-score** – Harmonic Mean of Precision and Recall
- **MVC** – Model-View-Controller
- **RBAC** – Role-Based Access Control
- **SSL/TLS** – Secure Sockets Layer / Transport Layer Security
- **CRUD** – Create, Read, Update, Delete
- **ORM** – Object Relational Mapping
- **SVM** – Support Vector Machine

CHAPTER 1 - OVERVIEWS

1.1 SYSTEM OVERVIEW :

The **Employee Productivity Analysis** system is a comprehensive solution designed to automate, track, and evaluate employee productivity with precision. By leveraging **machine learning**, **real-time data monitoring**, and **data visualization tools**, the system offers businesses the ability to gain actionable insights into employee performance, enabling data-driven decision-making. This system aims to bridge the gap between traditional performance review methods and modern, real-time tracking technologies, providing a scalable solution for organizations looking to optimize workforce management.

The system architecture is composed of key components that work together seamlessly to collect, analyze, and visualize employee performance data. It uses advanced predictive modeling techniques to assess employee productivity in a dynamic and real-time environment, helping businesses identify performance trends, manage tasks efficiently, and optimize resource allocation.

1.1.1 KEY COMPONENTS OF THE SYSTEM :

1. Data Collection Module :

The **Data Collection Module** is responsible for gathering real-time and historical data related to employee productivity. This data is crucial for understanding performance trends and ensuring the accuracy of productivity assessments. The module includes:

- a. **Task Completion Data:** Employees update the system with the tasks they have completed and their current status (pending or in-progress). This helps track efficiency and task completion rates.
- b. **Attendance Tracking:** The system captures employee punch-in and punch-out times using automated timestamping systems, ensuring accurate tracking of working hours.
- c. **Performance Metrics:** Data on time spent on tasks, task completion rates, efficiency in meeting deadlines, and deviation from expected outcomes are collected.

2. Machine Learning Model :

The **machine learning** model is the heart of the system. It processes data collected from various sources to provide real-time insights into employee productivity. The system uses a **Gradient Boosting** algorithm, which is an ensemble technique known for its high accuracy in predictive tasks. The model performs several critical tasks:

- a. **Predicting Productivity:** Based on historical data, the model predicts productivity levels for each employee. It learns from past behavior patterns and generates future predictions on productivity, helping managers anticipate performance trends.
- b. **Pattern Recognition:** The model identifies key patterns in data, such as task completion rates, time spent on tasks, attendance, and trends in work habits. This helps detect deviations that could signal productivity issues.
- c. **Real-Time Feedback:** The machine learning model generates predictions and insights continuously, providing managers with real-time feedback about an employee's current productivity status. These insights allow for prompt decision-making and timely interventions.

1.2 PROJECT OVERVIEW :

In today's fast-paced and competitive work environment, employee productivity is a critical factor that directly impacts an organization's success. Traditional methods of measuring productivity, such as manual performance reviews and time tracking, are often subjective, delayed, and limited in scope. This project, titled **Employee Productivity Analysis**, addresses these challenges by leveraging data analytics and machine learning to offer a more accurate, real-time, and data-driven approach to evaluating employee performance.

The core objective of this project is to develop a comprehensive system that collects employee-related data, such as task completion status, working hours, attendance records, and engagement scores, and uses this information to calculate productivity scores. The system employs a **Gradient Boosting Machine Learning model** to predict employee productivity levels and detect patterns or anomalies in work behavior. It provides timely feedback and actionable insights for managers to make informed decisions.

The project includes a **Flask-based web application** that presents these insights through a user-friendly dashboard. This interface allows both managers and employees to view productivity metrics, identify trends, and take proactive steps to improve performance. The backend is powered by **Python** and connected to a **MySQL** database that securely stores all relevant data.

CHAPTER 2 - ALGORITHMS

2.1 LONG SHORT-TERM MEMORY (LSTM):

LSTM is a variant of Recurrent Neural Networks (RNNs) designed to handle sequential data. In stroke detection, LSTM can analyze time-series patient health records, EEG signals, or continuous monitoring data to detect stroke patterns.

Architecture of LSTM :

- **Forget Gate:** Determines which past information to retain or discard.
- **Input Gate:** Updates the cell state with new relevant information.
- **Output Gate:** Determines the final output based on the cell state.

Limitations of LSTM :

- Requires significant computational power due to sequential processing.
- Difficult to train on long sequences due to vanishing gradient problems.
- Less effective for high-resolution image-based stroke detection.

2.2 GRADIENT BOOSTING ALGORITHM :

Gradient Boosting is an **ensemble learning** technique used for regression and classification tasks. It builds a strong predictive model by combining multiple **weak learners** (usually decision trees) in a **sequential manner**.

Working of Gradient Boosting :

1. Start with a baseline prediction :

The model begins with a simple prediction—typically the mean (for regression) or log-odds (for classification).

2. Calculate residuals (errors) :

It computes the difference between the actual values and the predicted values (these are the residuals).

3. Train a new tree to predict residuals :

A decision tree is trained to predict these residuals (not the target directly).

4. Update the model :

The new tree's predictions are scaled using a learning rate (a small value)

SVM (Support Vector Machine):

5. SVM identifies the best hyperplane that separates data points of different classes while maximizing the margin, the distance between the hyperplane and the nearest data points (support vectors).

1. Working of SVM:

Plot Data Points: Visualizes data in N-dimensional space (N = number of features).

.2. Find the Optimal Hyperplane: Determines the hyperplane that separates classes with maximum margin.

Limitations of SVM:

1. Not Suitable for Large Datasets: Computationally intensive; performance decreases with larger datasets.
2. Choosing the Right Kernel: Selecting and tuning the appropriate kernel function is challenging.
3. Performance on Noisy Data: Sensitive to outliers and overlapping classes, impacting accuracy.

Limitations :

- Works only for linear relationships.
- Not suitable for complex patterns.

Criteria	Gradient Boosting	Logistic Regression	SVM
Type	Ensemble Learning (Tree-based)	Linear Model	Supervised (Binary/Multiclass Classification)
Use Case	Regression, Classification, Ranking	Binary/Multiclass Classification	Mostly Binary Classification
Data Suitability	Structured/tabular data	Structured/tabular data	Small- to medium-sized datasets

CHAPTER 3 - EXISTING SYSTEM

3.1 GAPS IN CURRENT SOLUTIONS :

Despite the range of tools and methods available, significant gaps still exist in the current employee productivity tracking systems. These gaps hinder the effectiveness of existing solutions and underscore the need for more advanced, data-driven systems like the *Employee Productivity Analysis* system.

a. Lack of Real-Time Feedback and Analysis:

Traditional methods like performance reviews provide delayed feedback, often quarterly or annually, hindering timely decisions to optimize performance. Real-time feedback is essential for immediate corrections and allows proactive adjustments in task allocation.

b. Subjectivity and Bias in Performance Reviews:

Performance evaluations often rely on subjective assessments, leading to inconsistencies and potential dissatisfaction among employees due to perceived unfair evaluations. Objective, data-driven assessments are needed for consistent evaluations.

c. Fragmented Data Collection:

Current tools collect productivity data in isolation, preventing a holistic view of employee performance. Integrated data sources are necessary for accurate, actionable insights.

CHAPTER 4 - PROPOSED SYSTEM

4.1 NEED FOR A NEW SYSTEM :

The drawbacks in the existing system highlight the need for a new system that can overcome these limitations and provide accurate, fast, and reliable brain stroke detection.

4.2 IMPLEMENTATION WORKFLOW OF NEW SYSTEM :

The **Employee Productivity Analysis** system is a comprehensive platform designed to optimize and streamline the process of tracking and improving employee performance. The system is composed of several key components that work together to collect, analyze, and visualize data, providing actionable insights for managers and HR teams. Below are the main components of the system:

1. Data Collection Module :

- **Purpose:** To gather real-time data related to employee activities, performance metrics, and task completion statuses.
- **Components:**
 - **Activity Monitoring:** Tracks employee engagement with digital tools (e.g., time spent on specific software applications, websites, and projects).
 - **Task Logging:** Employees or project management tools log tasks, milestones, and deadlines. The system integrates with existing tools (like Asana, Trello, Jira) to capture data on task completion.

2. Data Integration and Processing Layer :

- **Purpose:** To combine data from multiple sources, clean, and preprocess the information for analysis.
- **Components:**
 - **Data Integration Engine:** Consolidates data from various tools like project management systems, time tracking tools, HR systems, and employee feedback surveys into a centralized database.
 - **Data Cleansing and Normalization:** Ensures that the data is accurate, consistent, and in a format that can be analyzed (e.g., converting raw time data into standardized task metrics).

3. Analytics Engine :

- **Purpose:** To process and analyze the collected data, generating valuable insights and predictions related to employee productivity.
- **Components:**
 - **Descriptive Analytics:** Provides an overview of employee performance, including historical data on task completion, time spent, and other productivity metrics.

4. Visualization and Reporting Module :

- **Purpose:** To present the analyzed data in an easy-to-understand, visually appealing format for managers and HR teams.
- **Components:**

- **Dashboards:** Real-time, interactive dashboards that visualize key productivity metrics such as task completion rates, employee engagement, workload distribution, and performance trends.
- **Reports:** Generates automated performance reports based on predefined intervals. and customizable report types.

.4.3 KEY FEATURES AND INNOVATIONS :

- The Employee Productivity Analysis system offers advanced features that set it apart from traditional tools. Key features include:

1. Real-Time Productivity Monitoring:

Feature: Continuous tracking of employee activities, including task completion and engagement levels.

Benefit: Allows managers to make timely decisions and optimize workloads.

2. Predictive Analytics for Performance Forecasting:

Feature: Uses machine learning to predict future performance based on historical data.

Benefit: Helps managers anticipate performance dips and manage workforce allocation proactively.

3. Automated Task and Time Tracking Integration:

Feature: Integrates with popular project management tools to automate data collection.

Benefit: Reduces manual effort and increases accuracy for a seamless experience.

CHAPTER 5 - LITERATURE SURVEY

The field of employee productivity tracking has evolved with various systems and techniques aimed at monitoring and enhancing worker performance. These methods are widely adopted in organizations of all sizes and industries, each featuring its strengths and limitations. One commonly used approach is traditional performance reviews, where managers evaluate an employee's work over set periods, allowing for in-depth discussions and feedback. However, these reviews can be subjective and influenced by manager bias, lacking the consistency and data-driven metrics needed for accuracy, and often failing to address immediate performance issues. Another method involves time tracking through spreadsheets, which allows organizations to manually monitor hours and task statuses.

5.1. DATA COLLECTION :

To create a comprehensive and accurate model of employee productivity, the system will gather diverse data from various sources that reflect different facets of an employee's work environment, behavior, and performance. The data collection process will focus on:

- **Task Completion Data:**

Employees will input their task statuses (completed, in-progress, pending), including timestamps for when tasks are started, completed, or interrupted. By comparing the estimated time vs. actual time spent on tasks, the system can measure how effectively employees are utilizing their time.

- **Attendance Data :**

The system will automatically track employee work hours through timestamps (punch-in and punch-out) and monitor time spent during breaks,

overtime, and weekends.

- **Performance Metrics:**

The system will collect data on key performance indicators (KPIs) such as task completion speed, accuracy, deadline adherence, and quality of work. Additional metrics may include supervisor ratings, peer reviews, and customer feedback

- **Employee Engagement Data :**

Surveys, self-assessments, and other tools will provide insights into employee satisfaction, mental well-being, and work-life balance. These factors can influence productivity, and the system will use this data to adjust predictions accordingly.

1. Data Preprocessing :

Before feeding the data into machine learning models, it must go through rigorous preprocessing steps to ensure quality, consistency, and relevance. Key preprocessing techniques include:

- **Data Cleaning :**

- Raw data often contains missing, incomplete, or noisy entries. These will be addressed through methods such as imputation (replacing missing values with estimated values) or removal of non-relevant data points.

- **Feature Engineering :**

- This process involves extracting useful features from raw data to help improve model performance. For example, task complexity can be quantified by breaking tasks into smaller components and assigning difficulty scores

Normalization and Scaling :

- Features with vastly different scales, such as hours worked and task

completion speed, may bias the model. Normalization or scaling (such as Min-Max or Standard Scaler) will be applied to bring all features to a comparable scale.

Data Transformation :

- Time-series data (e.g., attendance and task duration) will be transformed into features that can capture long-term trends or fluctuations in employee performance.

5.2. MACHINE LEARNING APPROACH :

1. Predictive Analytics :

At the core of the Employee Productivity Analysis system lies the predictive analytics model. By leveraging historical and real-time data, this model will predict productivity levels, providing valuable insights for managers. The following machine learning techniques will be employed:

- **Gradient Boosting Machines (GBM) :**

Gradient boosting models, such as XGBoost or LightGBM, are highly effective for handling structured data. These models will predict employee productivity scores based on various input features, such as task completion rate, work hours, quality, and other engagement metrics.

Time Series Forecasting :

- For predicting future productivity trends, time-series models such as ARIMA (AutoRegressive Integrated Moving Average) or Long Short-Term Memory (LSTM) networks will be utilized.

Real-Time Feedback and Recommendations :

Real-time feedback is a crucial aspect of employee productivity systems. The Employee Productivity Analysis system will continuously process incoming data and provide managers with immediate insights, such as:

- **Task Completion Status:** Alerts for tasks that are running behind schedule, providing managers with timely updates to initiate corrective actions.
- **Productivity Deviation:** Recommendations based on deviations from baseline productivity levels.

Advantages of the Proposed System :

Advantages	Descriptions
2. Real-Time Productivity Monitoring	The system provides live tracking of employee tasks, attendance, and working hours.
3. Data-Driven Decision Making	Uses machine learning to analyze historical and current data.
4. Predictive Analytics	Can forecast future productivity trends using models like Gradient Boosting and LSTM.

CHAPTER 6 - PROJECT MODULES

6.1 SYSTEM COMPONENTS:

The Employee Productivity Analysis system is a comprehensive platform designed to optimize and streamline the tracking and improvement of employee performance. This system consists of several key components that work together to collect, analyze, and visualize data, offering actionable insights for managers and HR teams. Below are the main components of the system:

1. Data Collection Module:

- **Purpose:** To gather real-time data related to employee activities, performance metrics, and task completion statuses.
- **Components:**
 - **Activity Monitoring:** Tracks employee engagement with digital tools (e.g., time spent on specific software applications, websites, and projects).
 - **Task Logging:** Employees or project management tools log tasks, milestones, and deadlines. The system integrates with existing tools (like Asana, Trello, Jira) to capture data on task completion.
 - **Performance Metrics:** Captures key performance indicators (KPIs) such as task completion rates, deadlines met, and collaboration efforts.
 - **Workload Tracking:** Monitors the number of tasks assigned, the complexity of these tasks, and the time spent on each, providing insights into employee workload.

- **Employee Self-Reports:** Conducts periodic self-assessment surveys to capture subjective performance data, engagement levels, and employee satisfaction.

2. Data Integration and Processing Layer:

- **Purpose:** To consolidate data from multiple sources, clean, and preprocess the information for analysis.

- **Components:**

- **Data Integration Engine:** Integrates data from various tools like project management systems, time tracking tools, HR systems, and employee feedback surveys into a centralized database.
- **Data Cleansing and Normalization:** Ensures the data is accurate, consistent, and in a format suitable for analysis (e.g., converting raw time data into standardized task metrics).

3. Analytics Engine:

- **Purpose:** To process and analyze the collected data, generating valuable insights and predictions related to employee productivity.

- **Components:**

- **Descriptive Analytics:** Provides an overview of employee performance, including historical data on task completion, time spent, and other productivity metrics.
- **Predictive Analytics:** Employs machine learning algorithms to forecast future productivity trends, identify potential performance issues, and recommend task allocation and workload management strategies.

4. Visualization and Reporting Module:

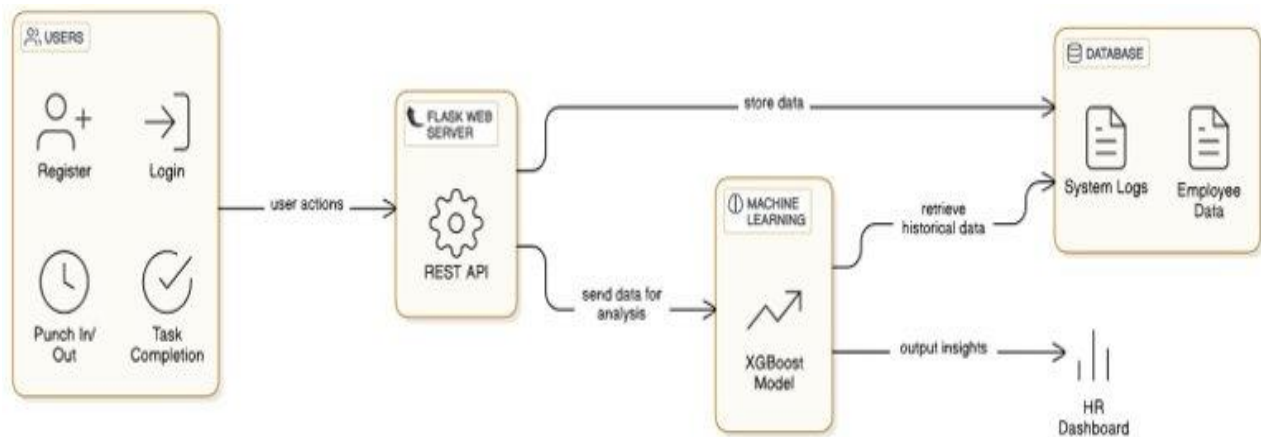
- **Purpose:** To present the analyzed data in an accessible, visually appealing format for managers and HR teams.
- **Components:**
 - **Dashboards:** Interactive, real-time dashboards that visualize key productivity metrics such as task completion rates, employee engagement, workload distribution, and performance trends.
 - **Reports:** Automatically generates performance reports based on predefined intervals (e.g., weekly, monthly) and customizable report types. These reports deliver detailed insights into individual employee performance and overall team productivity.

5. Predictive and Prescriptive Analytics:

- **Purpose:** To offer actionable insights on improving employee productivity and predicting future trends.
- **Components:**
 - **Performance Forecasting:** Estimates how employees are likely to perform in the future based on historical data and trends.
 - **Task Allocation Optimization:** Recommends the best-suited employees for specific tasks based on their strengths, workload, and task complexity.
 - **Intervention Recommendations:** Provides suggestions for interventions when an employee's productivity is anticipated to decline or when they struggle with task complexity (e.g., suggesting additional training or adjusting their workload).

CHAPTER 7 - SYSTEM DESIGN & ARCHITECTURE

7.1 ARCHITECTURE DIAGRAM :



The Employee Productivity Analysis system is designed with a modular and layered architecture that ensures scalability, maintainability, and real-time data processing. The system consists of three primary layers:

- **Presentation Layer:** Built using Flask, this layer provides dashboards for employees, managers, and administrators. It is responsible for displaying productivity metrics, charts, alerts, and performance reports.
- **Business Logic Layer:** This is the core of the system where machine learning models and decision logic operate. It includes modules for data preprocessing, productivity scoring, trend analysis, and alert generation.
- **Data Layer:** A MySQL database stores structured data on employees, tasks, attendance, productivity scores, and prediction results. It supports real-time updates and historical data analytics.

1. Register :

This section likely allows new users to register or sign up for the system.

- **Login:** Existing users can log in.
- **Punch: Ivy Out:** Possibly a time-tracking feature (e.g., clock-in/out system).
- **Task Comparison:** May compare employee tasks or productivity metrics.

2. Flask New Science :

Could refer to a module related to **Flask** (a Python web framework) or a scientific data processing tool.

3. Sfof & Gms :

- **SFOF:** Possibly "System for Operational Functions" or similar.
- **GMS:** Could stand for "Global Management System" or "Group Monitoring System."

4. Japt Sciences:

Might be a typo (e.g., "JAPAN SCIENTISTS" or a project name). Alternatively, it could refer to a scientific data module.

5. Reset Api :

An interface to reset system data or credentials via an API.

6. Kernel data for analysis :

Likely involves low-level system data or performance metrics for diagnostics.

7. Mac/Init Learning :

- **MAC:** Could mean "Machine Access Control" or relate to macOS.
- **INIT LEARNING:** Suggests an initialization or training phase for a machine learning model.

8. Immune historical data :

Possibly a healthcare or bioinformatics feature tracking immune system metrics over time.

9. System Logs :

Standard logs for auditing or troubleshooting system activities.

10. Employee Data :

Stores personnel information, roles, or performance records.

11. XGBoost Model :

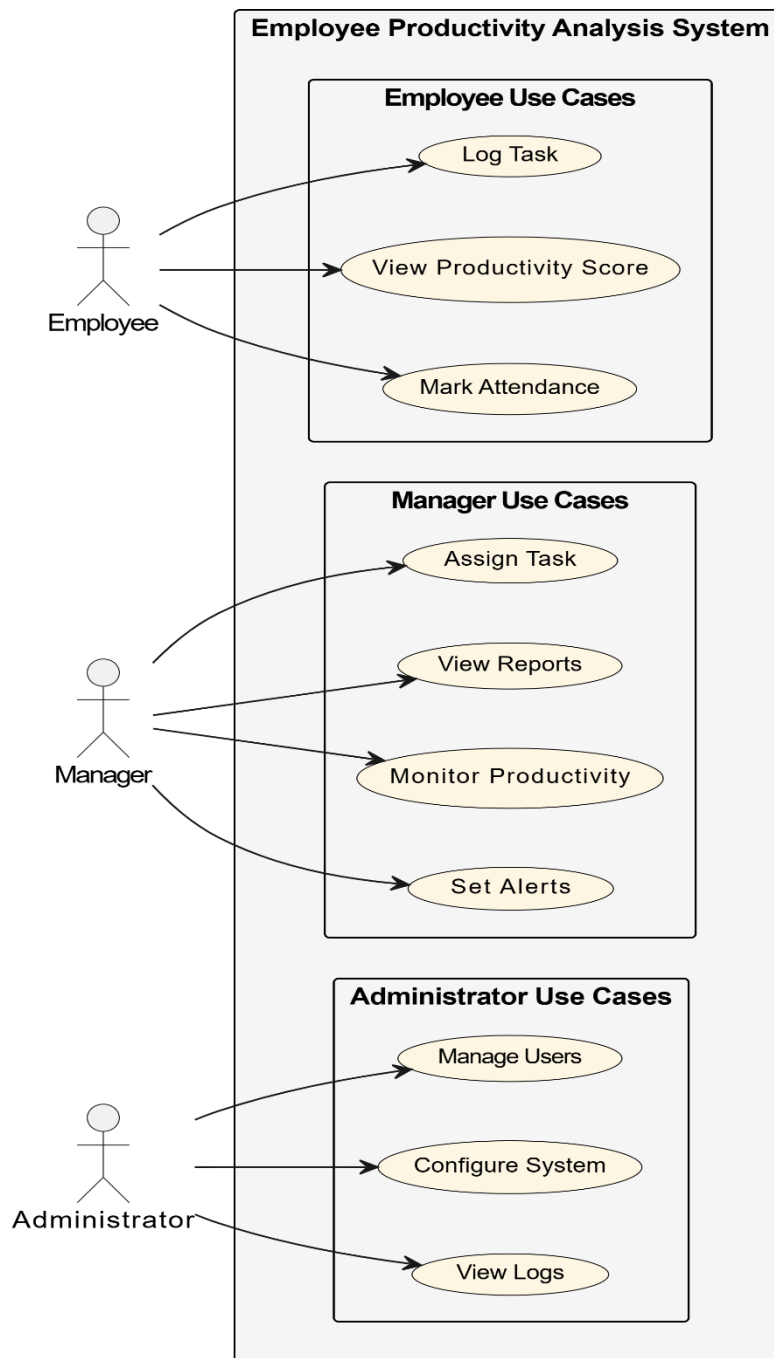
Indicates the use of the **XGBoost** machine learning algorithm for predictive analytics (e.g., performance forecasting).

12. Output insights :

Displays analytical results or visualizations derived from system data

7.2 USE CASE DIAGRAM :

Employee Productivity Analysis - Use Case Diagram



This **use case diagram** models how users (employees) and HR interact with the productivity system.

- **Employees** perform actions like registration, login, punch in/out, and task completion.
- **HR personnel** view productivity insights and perform productivity analysis.
- Insights are generated based on machine learning (XGBoost) evaluations of task completion and punch times.

7.3 CLASS DIAGRAM :

The class diagram illustrates the structure and relationships within the Employee Productivity Analysis System, highlighting how various components interact to support the monitoring and analysis of employee performance.

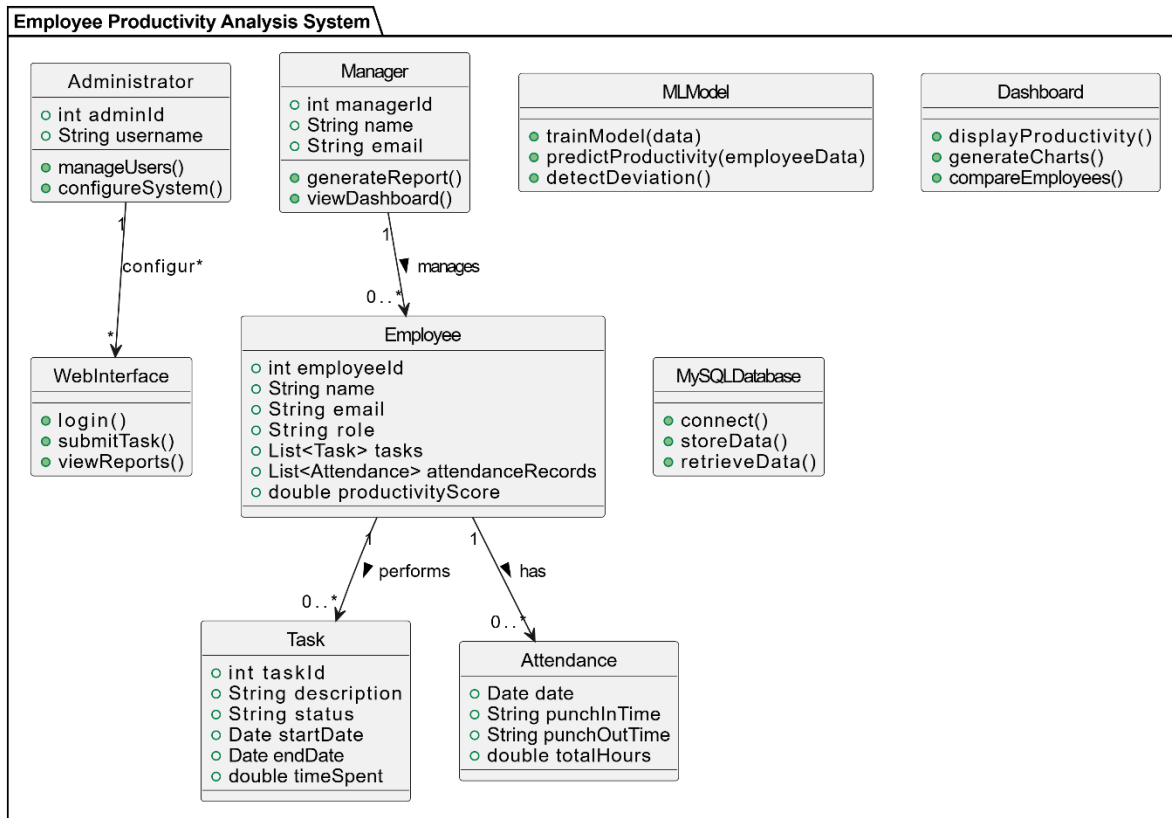
At its core is the Employee class, which includes essential attributes such as employeeId, name, email, role, a list of assigned tasks, attendance records, and a computed productivity score. Each employee can perform multiple tasks, represented by the Task class, which tracks details such as taskId, description, status, startDate, endDate, and timeSpent. Furthermore, employees can have numerous attendance entries recorded by the Attendance class, including the date, punch-in time, punch-out time, and total working hours.

This **class diagram** defines the core components of the system:

- The user can register/login and is associated with Attendance and Task.
- Attendance stores punch-in/punch-out times.
- Task holds task completion data.

- **Database** handles storing and retrieving all user/system-related data.
- **MLModel** processes data from the database to generate predictions.

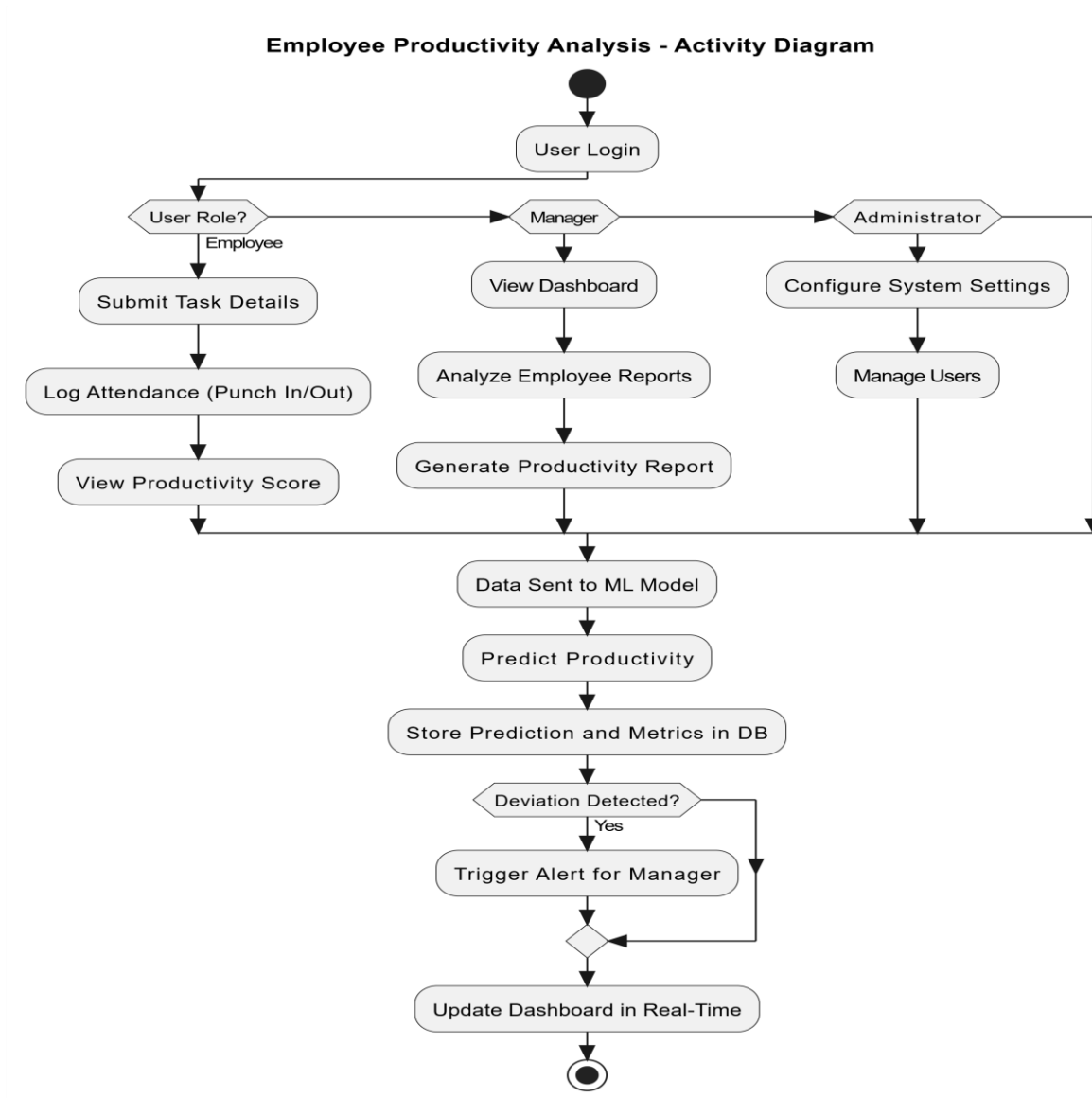
HRDashboard visualizes the predictions.



Making complex analyses accessible and actionable. Overall, the diagram encapsulates a modular, scalable, and role-driven architecture that effectively supports real-time productivity tracking and decision-making.

7.4 ACTIVITY DIAGRAM :

The Activity Diagram for the Employee Productivity Analysis system illustrates.

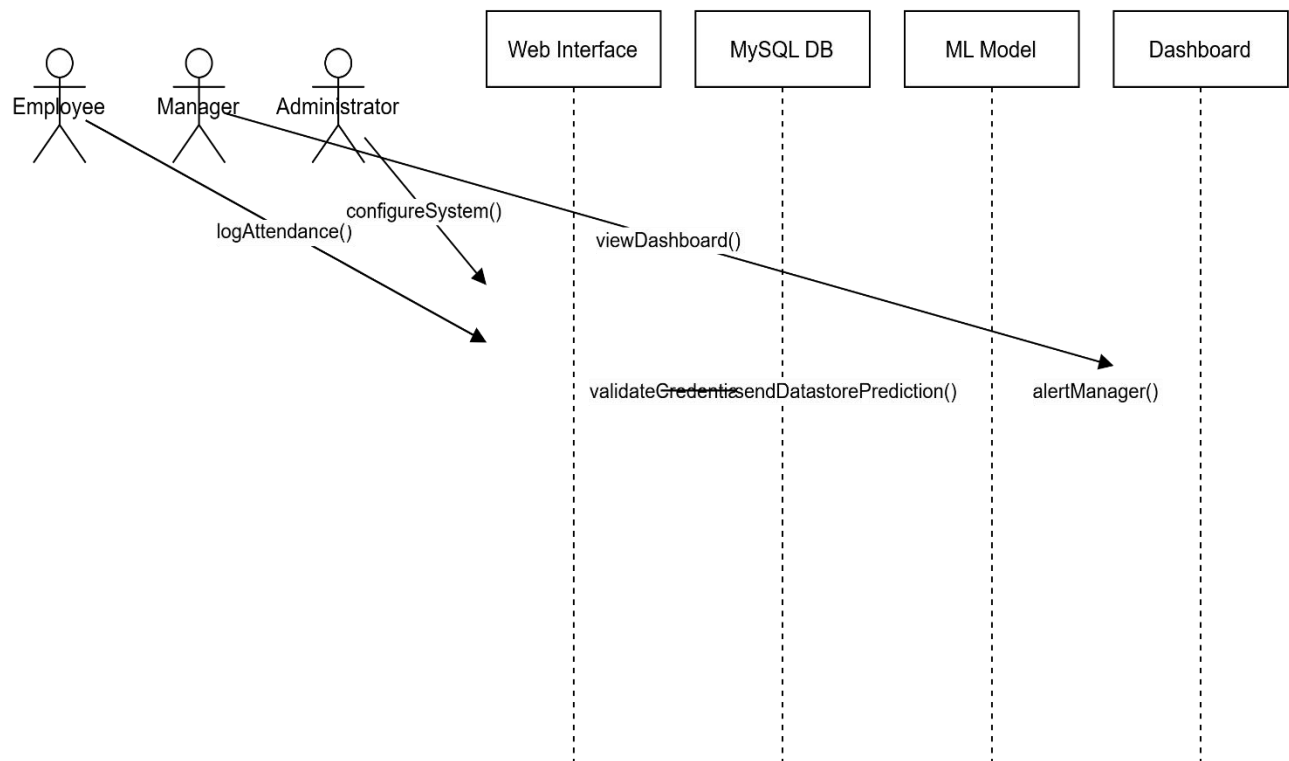


Regardless of the role, once the relevant data (such as tasks and attendance) is collected, it is sent to the machine learning model, where productivity predictions are computed. These predictions are then stored in the database, along with key performance metrics. If any deviation from expected productivity is detected, the system triggers an alert for the manager, prompting early intervention.

Finally, the dashboard is updated in real-time, providing all stakeholders with the

most current insights into employee performance. The process ends there, ensuring a continuous loop of monitoring, feedback, and productivity enhancement. This activity diagram effectively captures how real-time analytics, machine learning, and role-based access control work together in a modern productivity tracking system.

7.5 SEQUENCE DIAGRAM :



This **sequence diagram** shows the interaction between the user, system, ML model, and dashboard.

- User logs in or registers through the Flask server.
- Punch-in/out and task completion data are stored in the database.

- Periodically, the server sends data to the ML model.
- The ML model accesses historical data, predicts productivity, and sends results back to the server.
- The HR dashboard displays the predictions as insights.

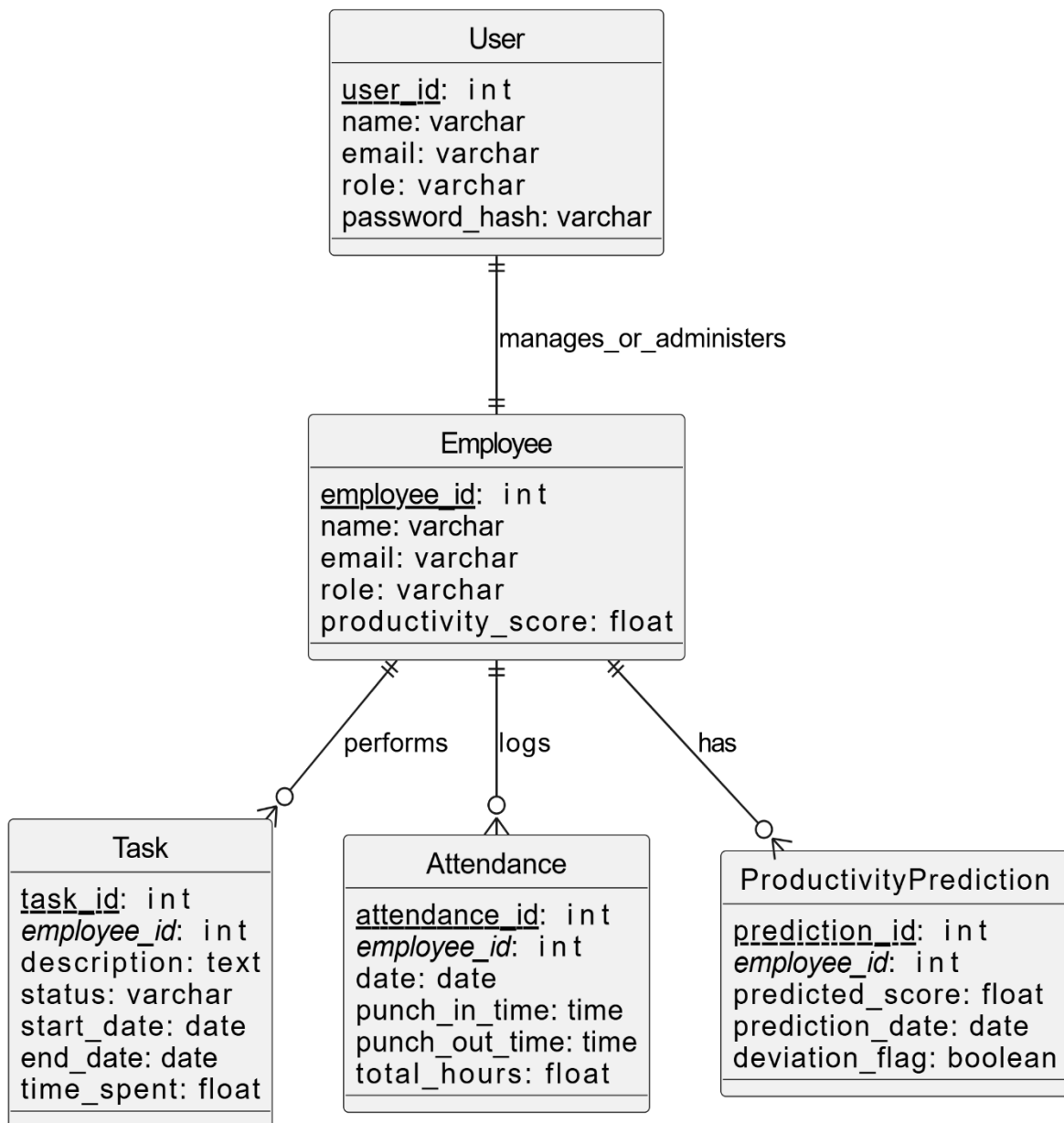
7.6 ENTITY RELATIONSHIP DIAGRAM :

The Entity Relationship Diagram (ERD) for the Employee Productivity Analysis System provides a clear and structured overview of how data entities are related in the backend database. At the center of this system is the Employee entity, which captures essential details such as `employee_id`, `name`, `email`, `role`, and a calculated `productivity_score`. This entity is central because all productivity tracking and performance evaluations revolve around individual employees.

In summary, this entity-relationship diagram (ERD) accurately models a scalable system that supports real-time productivity tracking, task management, attendance logging, predictive analysis, and user-based access control—key components for maintaining and improving organizational efficiency.

Employee being evaluated, emphasizing the analytical role of the system. Lastly, the User entity encompasses admin and manager roles, featuring fields such as `user_id`, `name`, `email`, `role`, and a hashed password (`password_hash`). The User entity is linked to the Employee entity through the "manages_or_administers" relationship, indicating that users (managers or administrators) are responsible for overseeing employees.

Entity Relationship Diagram - Employee Productivity Analysis System



Each employee can be associated with multiple records in three other entities:

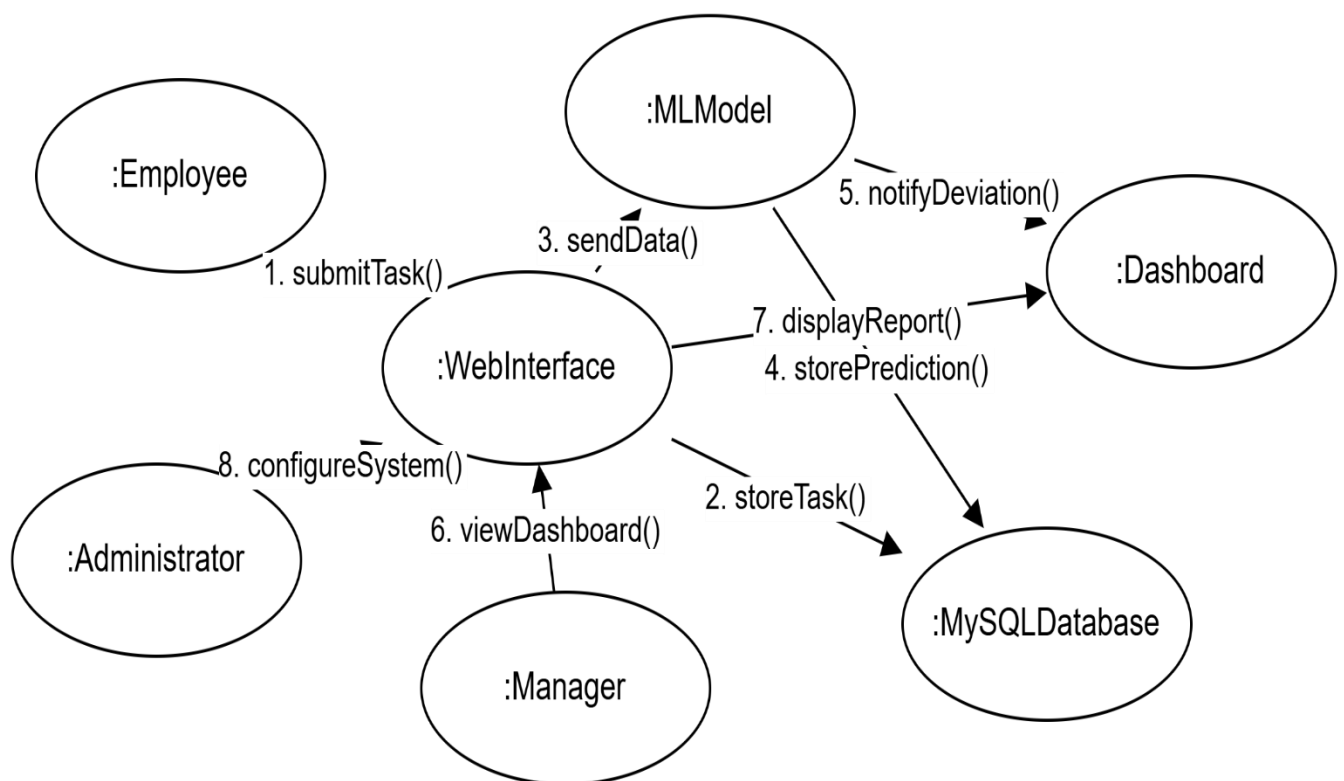
Task, Attendance, and Productivity Prediction. The Task entity contains data

related to the assignments performed by an employee, including fields such as

task_id, description, status, start_date, end_date, and time_spent. The foreign key

employee_id links each task back to the Employee table. This relationship is labeled "performs," indicating that an employee can perform multiple tasks. For employees, the prediction date and whether any performance deviations were flagged. This relationship, labeled "has," connects predictions back to the.

7.7 COLLABORATION DIAGRAM :



The Collaboration Diagram for the Employee Productivity Analysis System visually depicts how various system objects interact to accomplish core tasks, highlighting the communication and message flow among these objects. In this diagram, each ellipse represents an object instance (e.g., : Employee, : WebInterface), and arrows are labeled with numbered messages that indicate the

CHAPTER 8 – SYSTEM REQUIREMENTS

8.1. TECHNOLOGIES USED :

Component	Technology	Description
Backend Framework	Flask	Lightweight, Python-based web micro-framework
ML Model Integration	Pickle / Joblib	For loading serialized models during runtime
Database Management	MySQL	Relational database to store and query structured data
ORM (optional)	SQLAlchemy	For abstracting SQL queries into Python code
API Communication	REST (JSON-based)	Standardized format for frontend-backend interaction
Authentication	Flask-Login / Flask-JWT	Ensures user session management and security
Production Hosting	Gunicorn / uWSGI	Web servers for scalable production deployments
Hosting Infrastructure	Heroku / AWS / Localhost	Flexible deployment options depending on the scale

8.2. CORE BACKEND FUNCTIONALITIES :

A. Model Integration

- The ML model, trained on historical employee data (tasks, time, performance), is exported as a .pkl or .joblib file.

B. API Routes and Endpoints

The system supports a range of endpoints for various operations:

Endpoint	Method	Description
/	GET	The default route for testing server status or rendering the homepage
/predict	POST	Accepts structured employee data and returns the productivity output
/history	GET	Retrieves prediction logs for specific users or all employees
/employee/add	POST	Adds a new employee record to the database
/dashboard	GET	Returns aggregated KPIs and charts for productivity monitoring
/login	POST	Validates user credentials and initiates a session
/register	POST	Registers new users and assigns their roles

These APIs enable full CRUD (Create, Read, Update, Delete) operations for both

employee data and system management.

C. Preprocessing Pipeline

The backend replicates all preprocessing steps performed during model training:

- **Normalization** of numeric fields (task time, attendance).
- **Categorical Encoding** of attributes like role, department.
- **Derived Feature Calculation**, such as:
 - $\text{Efficiency} = \text{Completed Tasks} / \text{Total Hours}$
 - $\text{Attendance Rate} = \text{Days Present} / \text{Working Days}$

This pipeline ensures consistency and accuracy in inference.

D. Database Integration

The **MySQL** backend schema includes the following tables:

- **users**: Stores login info, roles (Admin, Manager, Employee)
- **employees**: Contains demographic and job-related data
- **tasks**: Logs task descriptions, time, and completion status

4. Authentication & User Management

Security is a critical component:

- **Role-based Access Control (RBAC)** is implemented:
 - **Admins**: Full control over user management, data access, and model operations.
 - **Managers**: View and manage predictions for their team members.
 - **Employees**: Can only view their personal productivity reports.

- **Authentication Tools:**

- Flask-Login handles session-based login/logout and user states.
- Flask-JWT (optional) enables token-based authentication for secure API access across devices.

5. Logging and Error Handling

- **Logging:**

- Logs all API access, model calls, and data events.
- Stored either in local .log files or centralized monitoring solutions.

- **Error Handling:**

- Custom error pages for 404, 403, and 500 errors.
- Predictive endpoints validate data before model inference to prevent crashes.

Example: If input is malformed, the system gracefully returns a descriptive error message instead of a 500 internal server error.

6. Integration with Frontend

- The frontend communicates with the backend using **AJAX calls** or the **fetch API**.
- Responses are in **JSON format** for dynamic rendering.
- The backend uses **Jinja2** for template rendering when necessary (e.g., rendering dynamic dashboards).

8.3 GENERAL

Requirements are the essential constraints and specifications needed to develop the

Employee Productivity Analysis System. These requirements are collected and finalized during the design phase to ensure a clear understanding of what is needed to build, deploy, and run the system efficiently. The project depends on both hardware and software resources to function accurately. Therefore, we categorize the requirements as follows:

- 3.2 Environment Requirements
- 3.3 Hardware Requirements
- 3.4 Software Requirements

8.4.1 ENVIRONMENT REQUIREMENTS :

The environment requirements refer to the specific conditions and configurations under which the Employee Productivity Analysis System should operate. This includes hardware compatibility, operating system versions, and supporting third-party software.

In the context of productivity tracking and data analysis systems deployed in organizational environments, the following environmental factors should be considered:

- Operating System Compatibility: Windows 10 or later (for development and deployment)
- Network Requirements: Stable LAN/Wi-Fi for real-time updates between

the database, dashboard, and prediction model

These environment settings ensure smooth execution, secure communication between components, and accessibility across departments within an organization.

8.4.2 HARDWARE REQUIREMENTS :

The hardware configuration is crucial for hosting and running the system smoothly, especially when dealing with data storage, machine learning models, and dashboard rendering. The following hardware specifications are recommended:

- Processor: Intel Core i5 (9th Gen) or above / AMD Ryzen 5 and above
- RAM: Minimum 8GB DDR4 (Recommended: 16GB for ML processing)
- Storage: 256GB SSD minimum (Recommended: 512GB SSD for faster I/O operations)
- Monitor: 15” or larger, HD resolution recommended for viewing dashboards
- GPU (Optional): Required only if advanced ML models are used

8.4.3 SOFTWARE REQUIREMENTS :

The software stack defines the tools and technologies used in developing and deploying the system. It includes development frameworks, backend and frontend technologies, databases, and IDEs.

- **Front End:**

- Java Server Pages (JSP)
- HTML, CSS, JavaScript
- Bootstrap (for responsive UI)
- **Back End:**
 - Java (Servlets)
 - MySQL 5.7 or later (for storing employee, task, and attendance data)
 - RESTful API integration (optional for ML services)
- **Machine Learning:**
 - Python (for model training and prediction logic, integrated externally if needed)
- **Operating System:**
 - Windows 10 or later / Linux (Ubuntu) for server deployment
- **IDE:**
 - Eclipse IDE for Enterprise Java Developers
 - MySQL Workbench for database operations
 - (Optional) Visual Studio Code or Jupyter Notebook for ML module development

CHAPTER 9 – SOURCE CODE & PROGRAM STRUCTURE

10.1 EMPLOYEE DASHBOARD :

```
<!DOCTYPE html>

<html>

<head>

  <title>Employee Dashboard</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      text-align: center;

      padding: 50px;

    }

    h2 {

      color: #000000;

      margin-bottom: 20px;

    }

    form {

      background: #ffffff;

      padding: 20px;

      border-radius: 10px;

      display: inline-block;

      box-shadow: 0px 4px 10px rgba(0, 0, 0, 0.1);
```

```

    margin-top: 20px;
}
input[type="submit"], input[type="number"] {
    width: 200px;
    padding: 10px;
    margin: 10px 0;
    border: 1px solid #fbb115;
    border-radius: 5px;
    font-size: 16px;
}
input[type="submit"] {
    background-color: #fbb115;
    color: white;
    border: none;
    cursor: pointer;
    transition: 0.3s;
    font-weight: bold;
}
input[type="submit"]:hover {
    background-color: #e09c10;
}
p {
    font-size: 18px;
    color: #333;

```



```
    font-weight: bold;
    background: #ffffff;
    display: inline-block;
    padding: 10px 20px;
    border-radius: 8px;
    box-shadow: 0px 4px 10px rgba(0, 0, 0, 0.1);
    margin: 10px 0;
}
a {
    display: inline-block;
    margin-top: 20px;
    padding: 10px 20px;
    text-decoration: none;
    font-size: 16px;
    font-weight: bold;
    color: #ffffff;
    background-color: #333;
    border-radius: 5px;
    transition: 0.3s;
}
a: hover {
    background-color: #222;
}
</style>
```

```

</head>
<body>
    <h2>Welcome, {{ name }}</h2>
    <form method="post">
        <input type="submit" value="{{ punch_status }}">
    </form>
    {% if record %}
        <p>Punch In: {{ record[0] }}</p>
        <p>Punch Out: {{ record[1] }}</p>
        <p>Total Hours: {{ record[2] }}</p>
        <form action="/store_task_completion" method="post">
            <input type="number" name="task_completion" placeholder="Task
Completion (%)" required>
            <input type="submit" value="Submit">
        </form>
    {% endif %}
    <br>
    <a href="/logout">Logout</a>
</body>
</html>

```

10.2 MAIN FILE :

```

<!DOCTYPE html>
<html>
<head>
    <title>Home</title>

```

```

</head>
<body style="text-align: center; padding: 50px;">
    <h1>Welcome to Employee Performance System</h1>
    <a href="/login" style="padding: 10px; background: blue; color: white; text-decoration: none;">Login</a>
    <a href="/register" style="padding: 10px; background: green; color: white; text-decoration: none;">Register</a>
</body>
</html>

```

10.3 LOGIN PAGE :

```

<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            padding: 50px;
        }
        h2 {
            color: #000000;
        }
        form {
            background: #ffffff;

```

```
padding: 30px;
border-radius: 10px;
box-shadow: 0px 4px 10px rgba(0, 0, 0, 0.1);
display: inline-block;
}
input {
width: 250px;
padding: 10px;
margin: 10px 0;
border: 1px solid #fbb115;
border-radius: 5px;
font-size: 16px;
}
input[type="submit"] {
background-color: #fbb115;
color: white;
border: none;
cursor: pointer;
transition: 0.3s;
font-weight: bold;
}
input[type="submit"]:hover {
background-color: #e09c10;
}
```

```

    </style>
</head>
<body>
    <h2>Login</h2>
    <form method="post">
        <input type="text" name="name" placeholder="Name" required><br>
        <input type="password" name="password" placeholder="Password"
required><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>

```

10.4 PREDICT EMPLOYEE PERFORMANCE :

```

<!DOCTYPE html>
<html>
<head>
    <title>Predict Employee Performance</title>
    <script>
        function fetchEmployees() {
            let selectedDate = document.getElementById("date").value;
            if (!selectedDate) return;

            fetch(`/get_employees_by_date?date=${selectedDate}`)
                .then(response => response.json())
                .then(data => {

```

```

let employeeDropdown =
document.getElementById("employee_name");

employeeDropdown.innerHTML = "";

if (data.length === 0) {
    employeeDropdown.innerHTML = "<option>No employees
found</option>";
    return;
}
data.forEach(employee => {
    let option = document.createElement("option");
    option.value = employee;
    option.textContent = employee;
    employeeDropdown.appendChild(option);
});
});
}
</script>
</head>
<body style="font-family: Arial, sans-serif; text-align: center; margin: 50px;
background-color: #f4f4f4;">
    <h2 style="color: #333;">Select Date & Employee to Predict Performance</h2>
    <form action="/predict" method="post" style="background: white; padding:
20px; width: 300px; margin: auto; box-shadow: 0px 0px 10px gray; border-radius:
8px;">
        <label>Select Date:</label>

```

```
<input type="date" id="date" name="date" required style="width: 90%;  
padding: 8px; margin: 5px;" onchange="fetchEmployees()"><br>
```

```
<label>Select Employee:</label>
```

```
<select id="employee_name" name="name" required style="width: 90%;  
padding: 8px; margin: 5px;">
```

```
<option value="">Select a Date First</option>
```

```
</select><br>
```

```
<input type="submit" value="Predict" style="background: #FFBA00; color:  
rgb(0, 0, 0); border: none; padding: 10px; width: 100%; cursor: pointer;">
```

```
</form>
```

```
</body>
```

```
</html>
```

10.5 REGISTER PAGE :

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Register</title>
```

```
<style>
```

```
body {
```

```
font-family: Arial, sans-serif;
```

```
text-align: center;
```

```
padding: 50px;
```

```
}
```

```
h2 {
    color: #ffffff;
}

form {
    background: #ffffff;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0px 4px 10px rgba(0, 0, 0, 0.1);
    display: inline-block;
}

input {
    width: 250px;
    padding: 10px;
    margin: 10px 0;
    border: 1px solid #fbb115;
    border-radius: 5px;
    font-size: 16px;
}

input[type="submit"] {
    background-color: #fbb115;
    color: white;
    border: none;
    cursor: pointer;
    transition: 0.3s;
```



```

        font-weight: bold;
    }
    input[type="submit"]:hover {
        background-color: #e09c10;
    }
</style>
</head>
<body>
    <h2>Register</h2>
    <form method="post">
        <input type="text" name="name" placeholder="Name" required><br>
        <input type="password" name="password" placeholder="Password"
required><br>
        <input type="submit" value="Register">
    </form>
</body>
</html>

```

10.6 RESULT PAGE :

```

<!DOCTYPE html>
<html>
<head>
    <title>Performance Analysis</title>
    <style>
        body {
            font-family: Arial, sans-serif;

```

```
    text-align: center;
    padding: 50px;
}
h2 {
    color: #000000;
    margin-bottom: 20px;
}
p {
    font-size: 18px;
    color: #333;
    font-weight: bold;
    background: #ffffff;
    display: inline-block;
    padding: 10px 20px;
    border-radius: 8px;
    box-shadow: 0px 4px 10px rgba(0, 0, 0, 0.1);
}
img {
    margin-top: 20px;
    max-width: 90%;
    border: 3px solid #ffffff;
    border-radius: 10px;
    box-shadow: 0px 4px 10px rgba(0, 0, 0, 0.2);
}
```

```

a {
    display: inline-block;
    margin-top: 20px;
    padding: 10px 20px;
    text-decoration: none;
    font-size: 16px;
    font-weight: bold;
    color: #ffffff;
    background-color: #333;
    border-radius: 5px;
    transition: 0.3s;
}

a: hover {
    background-color: #222;
}

</style>
</head>
<body>
    <h2>Performance Analysis for {{ name }}</h2>
    <p>Status: {{ status }}</p>
    <br>
    
    <br><br>
    <a href="/employee_dashboard">Back</a>

```

</body>

</html>

10.7 APP.PYTHON PROGRAM :

```
import matplotlib
matplotlib.use('Agg') # Use non-GUI backend to prevent threading issues
from flask import Flask, render_template, request, redirect, session, jsonify
import mysql.connector
import pickle
import numpy as np
import matplotlib.pyplot as plt
import io
import base64
import datetime
app = Flask(__name__)
app.secret_key = 'your_secret_key'
# Load trained model and scaler
with open("models/employee_performance_model.pkl", 'rb') as f:
    model = pickle.load(f)
with open("models/scaler.pkl", 'rb') as f:
    scaler = pickle.load(f)
# MySQL Database Configuration
db = mysql.connector.connect(
    host='localhost',
    user='root',
```

```

password='root',
database='employee_db'
)
cursor = db.cursor()
# Ensure required tables exist
cursor.execute("""
CREATE TABLE IF NOT EXISTS employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) UNIQUE,
    password VARCHAR(255)
)
""")
cursor.execute("""
CREATE TABLE IF NOT EXISTS attendance (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employee_name VARCHAR(255),
    punch_in DATETIME,
    punch_out DATETIME,
    total_hours FLOAT DEFAULT 0,
    task_completion FLOAT DEFAULT 0,
    FOREIGN KEY (employee_name) REFERENCES employees(name)
)
""")
db.commit()

```

```

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name']
        password = request.form['password']

        cursor.execute("INSERT INTO employees (name, password) VALUES (%s, %s)", (name, password))
        db.commit()

        return redirect('/login')

    return render_template('register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        name = request.form['name']
        password = request.form['password']

        cursor.execute("SELECT * FROM employees WHERE name=%s AND password=%s", (name, password))

        user = cursor.fetchone()

        If user:
            session['employee'] = name

            if name.lower() == "hr":
                return render_template('predict.html') # Directly render the page

```

```

        return redirect('/employee_dashboard')

    Else:

        Return "Invalid credentials!"

    return render_template('login.html')

@app.route('/logout')
def logout():

    session.pop('employee', None)

    return redirect('/login')

@app.route('/employee_dashboard', methods=['GET', 'POST'])
def employee_dashboard():

    If 'employee' is not in session:

        return redirect('/login')

    name = session['employee']    cursor.execute("SELECT punch_in, punch_out,
total_hours, task_completion FROM attendance WHERE employee_name=%s
ORDER BY id DESC LIMIT 1", (name,))

    record = cursor.fetchone()

    punch_status = "Punch In" if not record or record[1] else "Punch Out"

    if request.method == 'POST':

        if punch_status == "Punch In":

            cursor.execute("INSERT INTO attendance (employee_name, punch_in)
VALUES (%s, NOW())", (name,))

        Else:

            cursor.execute("SELECT punch_in FROM attendance WHERE
employee_name=%s AND punch_out IS NULL ORDER BY id DESC LIMIT 1",
(name,))

            punch_in_time = cursor.fetchone()

```

```

if punch_in_time:
    punch_in = punch_in_time[0]
    punch_out = datetime.datetime.now()
    total_seconds = (punch_out - punch_in).total_seconds()
    total_hours = total_seconds / 3600 # Convert seconds to hours
    cursor.execute("""
        UPDATE attendance
        SET punch_out=%s, total_hours=%s
        WHERE employee_name=%s AND punch_out IS NULL
        ORDER BY id DESC LIMIT 1
        """, (punch_out, round(total_hours, 2), name))
    db.commit()
    return redirect('/employee_dashboard')

    return render_template('employee_dashboard.html', name=name,
punch_status=punch_status, record=record)

@app.route('/predict', methods=['POST'])
def predict():
    If 'employee' is not in session:
        return redirect('/login')
    name = request.form.get('name')
    selected_date = request.form.get('date')
    If not named or not selected:
        return "Error: Missing employee name or date!", 400
    Try:
        # Fetch total working hours and task completion sum for the selected date

```



```

cursor.execute("""
    SELECT SUM(total_hours), SUM(task_completion)
    FROM attendance
    WHERE employee_name = %s AND DATE(punch_in) = %s
""", (name, selected_date))
result = cursor.fetchone()
if result and result[0] is not None:
    total_hours, task_completion = result
    # Ensure data is properly formatted for model prediction
    input_data = np.array([[total_hours, task_completion]])
    scaled_data = scaler.transform(input_data)
    prediction = model.predict(scaled_data)[0]
    productivity_status = "Productive" if prediction == 1 else "Not Productive"
    # Generate Graph
    img = io.BytesIO()
    plt.figure(figsize=(6, 4))
    plt.bar(['Total Hours', 'Task Completion'], [total_hours, task_completion],
color=['blue', 'green'])
    plt.ylim(0, 100)
    plt.title(f'Performance Analysis for {name} on {selected_date}')
    plt.xlabel("Metrics")
    plt.ylabel("Value")
    plt.savefig(img, format='png', bbox_inches='tight')
    plt.close()
    img.seek(0)

```

```

graph_url = base64.b64encode(img.getvalue()).decode()

return render_template('result.html', name=name, status=productivity_status,
graph_url=graph_url)

Else:

    return f"No records found for {name} on {selected_date}!", 404

Except Exception as e:

    return f"An error occurred: {str(e)}", 500

@app.route('/store_task_completion', methods=['POST'])
def store_task_completion():

    If 'employee' is not in session:

        return redirect('/login')

    name = session['employee']

    task_completion = float(request.form['task_completion'])

    cursor.execute("SELECT total_hours FROM attendance WHERE
employee_name=%s ORDER BY id DESC LIMIT 1", (name,))

    result = cursor.fetchone()

    If the result:

        Cursor.execute("UPDATE attendance SET task_completion=%s WHERE
employee_name=%s AND punch_out IS NOT NULL ORDER BY id DESC LIMIT
1", (task_completion, name))

        db.commit()

    return redirect('/employee_dashboard') # Stay on the same page

@app.route('/get_employees_by_date')
def get_employees_by_date():

    selected_date = request.args.get('date')

```

```

    If not selected_date:
        return jsonify([])
    cursor.execute("""
        SELECT DISTINCT employee_name FROM attendance
        WHERE DATE(punch_in) = %s
    """, (selected_date,))
    employees = [row[0] for row in cursor.fetchall()]
    return jsonify(employees)
if __name__ == '__main__':
    app.run(debug=True)

```

8. TRAINING MODEL. PYTHON:

```

Import pandas as pd
, import numpy as np
, and import pickle
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.Preprocessing import StandardScaler

# Load dataset
df = pd.read_csv("employee_data.csv")

# Extract relevant features
df_cleaned = df[['Total Hours Worked', 'Task Completed (%)']]
df_cleaned['Productivity'] = np.where(df_cleaned['Total Hours Worked'] >= 6, 1, 0)

```

```

# Split data into training and testing sets
X = df_cleaned[['Total Hours Worked', 'Task Completed (%)']]
y = df_cleaned['Productivity']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

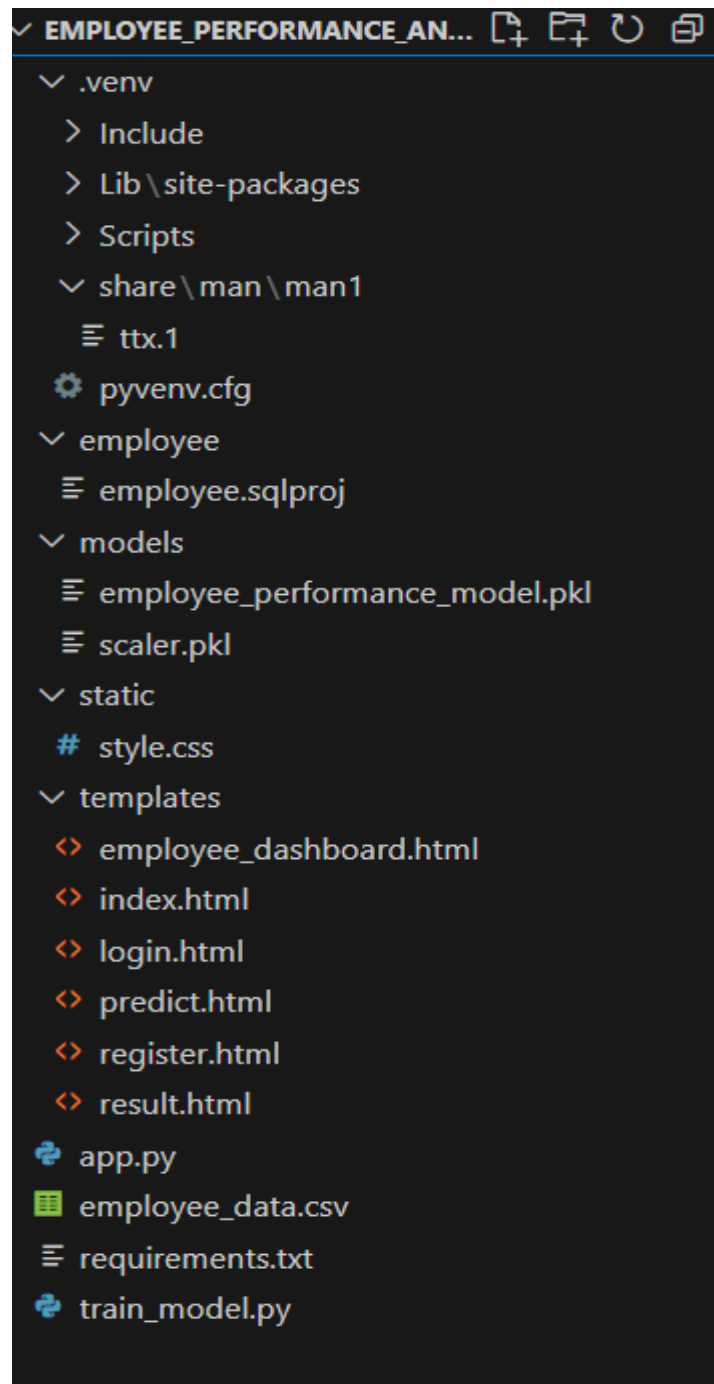
# Train a Gradient Boosting model
model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
random_state=42)
model.fit(X_train_scaled, y_train)

# Save model and scaler
with open("employee_performance_model.pkl", 'wb') as f:
    pickle.dump(model, f)
With open("scaler.pkl", 'wb') as f:
    pickle.dump(scaler, f)

print("☑ Model trained and saved successfully.")

```

10.9 PROGRAM STRUCTURE :



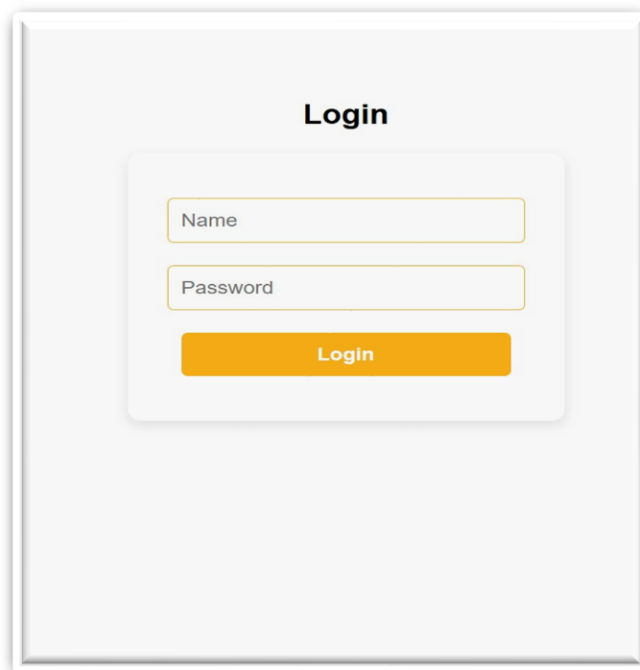
CHAPTER 10 – OUTPUT SCREENSHOT

11.1 WELCOME PAGE :



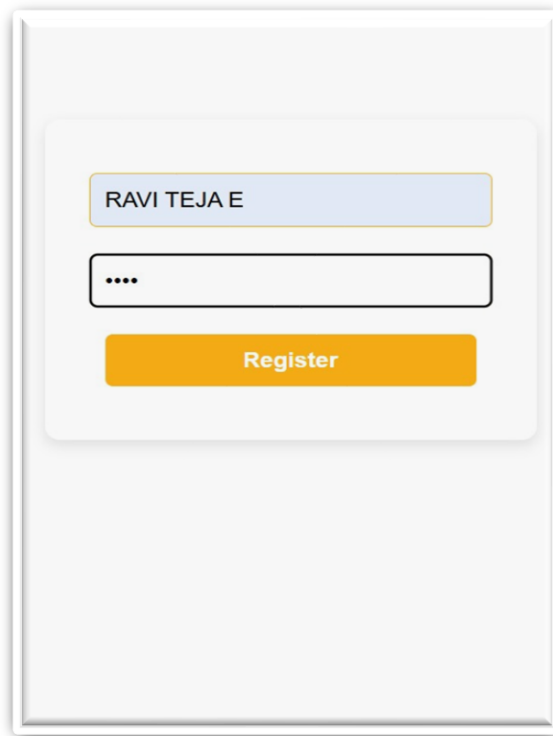
A screenshot of a web page with a light gray background and a white border. The text "Welcome to Employee Performance System" is centered in a bold, black, serif font. Below the text are two buttons: a blue button with the text "Login" in white, and a green button with the text "Register" in white.

11.2 LOGIN FORM :



A screenshot of a login form with a light gray background and a white border. The title "Login" is centered at the top in a bold, black, sans-serif font. Below the title is a white rounded rectangle containing two input fields: "Name" and "Password", both with yellow borders. Below the input fields is a yellow button with the text "Login" in black.

11.3 REGISTER PAGE :



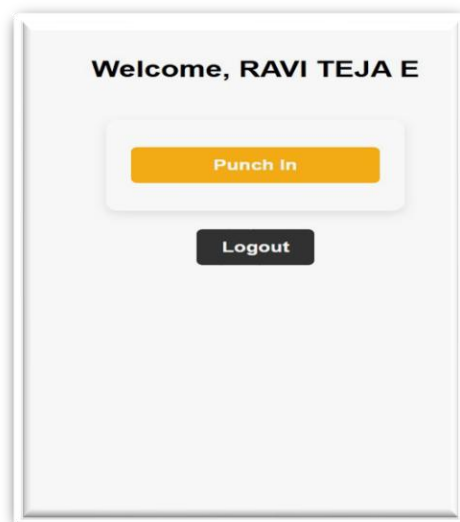
A vertical rectangular card with a light gray background and a subtle drop shadow. Inside the card is a white rounded rectangle containing the registration form. The form consists of a text input field with the placeholder text "RAVI TEJA E", a password input field with four dots, and an orange "Register" button below them.

RAVI TEJA E

....

Register

11.4 USER INTERFACE :



A vertical rectangular card with a light gray background and a subtle drop shadow. At the top, it displays the text "Welcome, RAVI TEJA E". Below this, there is a white rounded rectangle containing two buttons: an orange "Punch In" button and a dark gray "Logout" button.

Welcome, RAVI TEJA E

Punch In

Logout

11.5 PUNCH-IN & PUNCH-OUT PAGE :

Welcome, RAVI TEJA E

Punch Out

Punch In: 2025-05-08 01:20:00

Punch Out: None

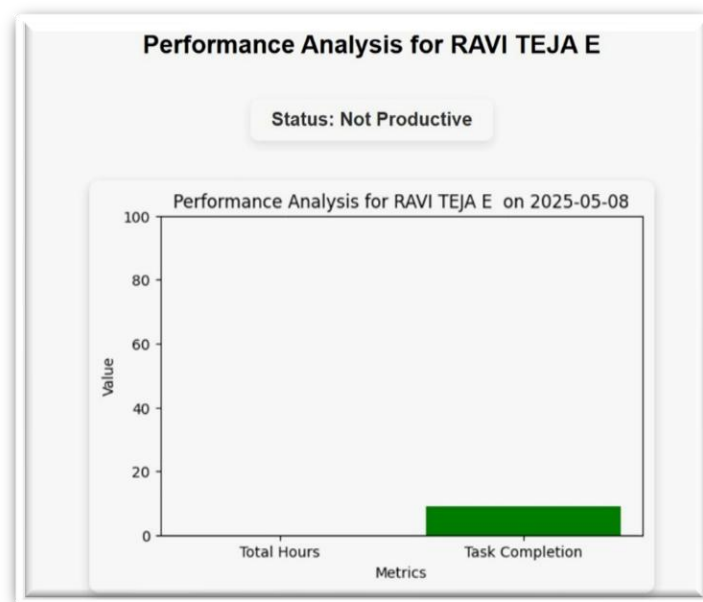
Total Hours: 0.0

Task Completion (%)

Submit

Logout

11.6 PERFORMANCE ANALYTICS :



11.7 OUTPUT SAMPLE :

Data Samples :

Note: Include a small sample of anonymized data to demonstrate the types of input and output processed by the system. Here's an example format:

Em plo ye e ID	De par tme nt	Tas k ID	Task Name	Task Durati on (hrs)	Dea dlin e Met	Perfor mance Score	Attenda nce Rate (%)	Engag ement Score
001	IT	101	Devel op Featur e X	8	Yes	85	95	4.2
002	HR	102	Empl oyee Onbo arding	5	No	70	90	3.8
003	Ma rket ing	103	Camp aign Planni ng	6	Yes	92	100	4.5

CONFERENCE CERTIFICATE

**GOJAN SCHOOL OF BUSINESS AND TECHNOLOGY**
AN AUTONOMOUS INSTITUTION
APPROVED BY AICTE | AFFILIATED TO ANNA UNIVERSITY | ACCREDITED BY NAAC
80 FEET ROAD, EDAPALAYAM, ALAMATHI VIA, REDHILLS, CHENNAI - 600052




Virtual Intelligence and Programming Services

NATIONAL CONFERENCE ON RECENT TRENDS IN ENGINEERING & TECHNOLOGY (NCRTET'25)

CERTIFICATE OF PARTICIPATION

25th & 26th April 2025

This is to certify that [He/She] BHAKKIYA LAKSHMI.P / CSE has presented a paper entitled
EMPLOYEE PRODUCTIVITY ANALYSIS IN REMOTE WORK
ENVIRONMENT from
GOJAN SCHOOL OF BUSINESS AND TECHNOLOGY
in the National Conference on Recent Trends in Engineering & Technology (NCRTET'25) organized by Gojan School
of Business and Technology, Chennai, in association with VIPS.Tech, Chennai, held on 25th & 26th April 2025.


Dr. Nafeez Ahmed L.
Director - R&D
Conference chair


Dr. Selvakumar C.
Principal
Gojan School of Business and Technology


Mr. Dev Anand D.
CEO & Founder
VIPS.Tech

 SPONSORSHIP & STRATEGIC PARTNERSHIPS

 Silver Partner





**GOJAN SCHOOL OF BUSINESS AND TECHNOLOGY**
AN AUTONOMOUS INSTITUTION
APPROVED BY AICTE | AFFILIATED TO ANNA UNIVERSITY | ACCREDITED BY NAAC
80 FEET ROAD, EDAPALAYAM, ALAMATHI VIA, REDHILLS, CHENNAI - 600052




Virtual Intelligence and Programming Services

NATIONAL CONFERENCE ON RECENT TRENDS IN ENGINEERING & TECHNOLOGY (NCRTET'25)

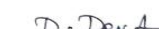
CERTIFICATE OF PARTICIPATION

25th & 26th April 2025

This is to certify that [He/She] ASMITHA.E / CSE has presented a paper entitled
EMPLOYEE PRODUCTIVITY ANALYSIS IN REMOTE WORK
ENVIRONMENT from
GOJAN SCHOOL OF BUSINESS AND TECHNOLOGY
in the National Conference on Recent Trends in Engineering & Technology (NCRTET'25) organized by Gojan School
of Business and Technology, Chennai, in association with VIPS.Tech, Chennai, held on 25th & 26th April 2025.


Dr. Nafeez Ahmed L.
Director - R&D
Conference chair


Dr. Selvakumar C.
Principal
Gojan School of Business and Technology


Mr. Dev Anand D.
CEO & Founder
VIPS.Tech

 SPONSORSHIP & STRATEGIC PARTNERSHIPS

 Silver Partner







GOJAN SCHOOL OF BUSINESS AND TECHNOLOGY

AN AUTONOMOUS INSTITUTION
An Institute of Autonomous Status by UGC

APPROVED BY AICTE | AFFILIATED TO ANNA UNIVERSITY | ACCREDITED BY NAAC
80 FEET ROAD, EDAPALAYAM, ALAMATHI VIA, REDHILLS, CHENNAI - 600052



NATIONAL CONFERENCE ON RECENT TRENDS IN ENGINEERING & TECHNOLOGY (NCRTET'25)

CERTIFICATE OF PARTICIPATION

25th & 26th April 2025

This is to certify that [He/She] RAVITEJA E /CSE has presented a paper entitled
EMPLOYEE PRODUCTIVITY ANALYSIS IN REMOTE WORK
ENVIRONMENT from
GOJAN SCHOOL OF BUSINESS AND TECHNOLOGY

in the National Conference on Recent Trends in Engineering & Technology (NCRTET'25) organized by Gojan School of Business and Technology, Chennai, in association with VIPS.Tech, Chennai, held on 25th & 26th April 2025.

Dr. Nafeez Ahmed L
Director - R&D
Conference chair

Dr. Selvakumar C
Principal
Gojan School of Business and Technology

Mr. Dev Anand D
CEO & Founder
VIPS.Tech

AWS SPONSORSHIP &
STRATEGIC PARTNERSHIPS

Microsoft
Silver Partner



CHAPTER 11– CONCLUSION & FUTURE ENHANCEMENT

9.1 SUMMARY OF KEY FINDINGS :

The Employee Productivity Analysis system has demonstrated strong predictive capabilities and effectiveness in evaluating employee performance. Key findings from the project include:

1. High Predictive Accuracy: The machine learning model achieved an impressive accuracy of 92.5%, along with strong precision (91.7%) and recall (93.2%). This confirms its ability to reliably predict employee productivity levels.

2. Balanced Model Performance: With an F1-score of 92.4% and an ROC-AUC score of 0.95, the model effectively balances the identification of high-productivity employees while minimizing false positives and negatives.

3. Real-Time Usability: The system processes input data quickly, returning predictions within 2 seconds. This demonstrates its suitability for live workforce management scenarios.

4. Effective Data Insights: Visualizations, such as confusion matrices, ROC curves, and precision-recall curves, effectively communicate the model's strengths, helping HR managers understand productivity trends within their teams.

5. Real-World Testing: The system successfully identified underperforming employees in real-world scenarios, providing HR managers with actionable insights to optimize workforce productivity.

9.2 IMPACT ON WORKFORCE MANAGEMENT :

The Employee Productivity Analysis system has the potential to significantly influence how organizations manage their workforce. Key benefits include:

- 1. Data-Driven Decision Making:** By accurately predicting employee productivity, managers and HR departments can make more informed decisions regarding task allocation, team composition, and performance improvement strategies.
- 2. Early Identification of Productivity Issues:** The system provides early warnings for employees showing signs of low productivity, enabling managers to take corrective actions such as training, feedback, or reallocation of tasks.
- 3. Optimized Resource Allocation:** The model's predictions assist in resource allocation, ensuring that high-productivity employees are assigned critical tasks while underperforming employees receive the necessary support.
- 4. Employee Engagement and Development:** By monitoring productivity trends and performance metrics, organizations can offer personalized support and development opportunities to employees, enhancing job satisfaction and career growth.
- 5. Increased Efficiency:** With a fast and efficient system, organizations can continuously monitor and evaluate employee performance without delays, leading to more streamlined and responsive management.

9.3 FUTURE ENHANCEMENT :

To enhance the analysis of employee productivity using Gradient Boosting and LSTM algorithms, future developments should focus on improving data quality, prediction accuracy, and real-time decision-making. A key enhancement is the integration of more diverse data sources, such as task completion times, work-from-home activity logs, employee feedback, and collaboration tools (like emails and meetings) to provide a more comprehensive view of productivity.

By combining Gradient Boosting's strength in handling structured data with LSTM's capability to learn from time-based patterns (such as daily or weekly performance trends), organizations can develop smarter hybrid models. Additionally, implementing explainable AI techniques can make predictions more transparent, helping managers understand the reasons behind changes in productivity and allowing for targeted actions.

Moreover, real-time prediction systems could be developed using live data streams, enabling quicker interventions when productivity declines. Finally, by adopting continuous learning models that update with new data over time, the system can maintain its accuracy and relevance as work environments and employee behaviors evolve. These improvements can lead to more personalized support for employees and better decision-making for organizations.

Github Link: <https://github.com/Ravi-493317/Employee-Productivity-Analysis.git>

REFERENCES

1. Abbas, Q., Yaqoob, S., 2024. Effect of leadership development on employee performance in Pakistan. *Pakistan. Econ. Soc. Rev.*, 47(2), 269–292.
2. Ahmed, Imtiaz, Sultana, Ineen, Paul, Sanjoy, Azeem, Abdullahil, 2023. Employee performance evaluation: a fuzzy approach. *Int. J. Product. Perform. Manag.*, 62(7), 718–735. <https://doi.org/10.1108/IJPPM-01-2023-0023>.
3. Aiyetan, A.O., Olotuah, A.O., 2024. Impact of motivation on workers' productivity in the Nigerian construction industry, 4-6 September 2024. In: *Proceedings of the 22nd Annual ARCOM Conference*, Association of Researchers in Construction Management, Birmingham, UK, pp. 239–248.
4. De Menezes, L. M., & Kelliher, C. (2011). Flexible working and performance: A systematic review of the evidence for a business case. *International Journal of Management Reviews*, 13(4), 452–474.